Team members:

Anushri Neramballi Raghavendra (UF ID: 38155612)
Avantika Holla Sathyanarayana (UF ID: 25363524)

Demo Link

Program Structure:

- The Reddit Clone simulation initializes by creating a central Reddit Engine actor that coordinates all system activities and manages data for users, subreddits, posts, comments, and messages.

- Based on user-provided input parameter: num_users, num_subreddits, and simulation_time, the simulator spawns multiple Client actors, each representing an independent Reddit user.

- The Reddit Engine handles global operations such as account creation, subreddit management, and message routing. Each client interacts with the Reddit Engine through defined message types that model user actions.

- The simulation runs recursively via the simulate_activity procedure until the specified simulation time expires. Throughout this time, clients randomly perform actions such as joining or leaving subreddits, creating posts or comments, voting, or sending messages.

- Once the simulation completes, the Reddit Engine aggregates and prints system statistics, summarizing total users, subreddits, posts, comments, messages, and vote counts.

What is Working:

The Reddit Clone implementation successfully supports all major Reddit-like interactions, including account registration, subreddit creation and membership management, post and comment hierarchies, upvotes/downvotes with karma computation, direct messaging, and personalized feed retrieval.

The simulator dynamically models user behavior under varying system scales, correctly handling concurrent actions and maintaining data consistency across users and subreddits. It also demonstrates realistic workload patterns by using a Zipf distribution for subreddit popularity, ensuring that certain communities become highly active with more posts and interactions, while others remain relatively quiet, mimicking real social media dynamics.

During testing, the system was observed to scale efficiently across a large number of users and subreddits, with appropriate distribution of activity and stable performance across different simulation durations.

Implemented Functionality and Messages:

Account Management:

- Message: AddAccount
  - Registers a new user in the Reddit Engine.
  - Initializes an Account record containing username, password, karma, an inbox (for direct messages), and a list of subscribed subreddits.

- Message: ComputeKarma
  - Calculates the user's total karma based on the upvotes and downvotes on their posts.
  - Increments karma for every upvote received and decrements it for every downvote.

Subreddit Management:

- Message**:** CreateSubReddit
  - Creates a new subreddit with a unique name and an empty member list.
  - Adds it to the global list of subreddits managed by the Reddit Engine.

- Message**:** JoinSubReddit
  - Allows a valid user to join an existing subreddit.
  - Ensures both the user and subreddit exist before updating membership lists.

- Message**:** LeaveSubReddit
  - Enables a user to leave a subreddit.
  - Performs validation and removes the user from the subreddit's member list.

Posting and Commenting:

- Message: CreatePost
  - Validates the existence of both user and subreddit before allowing a post.
  - Creates a Post object containing author, content, timestamp, upvotes, downvotes, and a list of comments.
  - The post is appended to the subreddit's post list.

- Message: CreateComment
  - Selects a random post in a subreddit and adds a comment under it.
  - Each Comment object stores author, content, timestamp, and a nested list of replies.

- Message: CreateCommentReply
  - Enables hierarchical commenting by allowing replies to existing comments on a post.

Voting and Karma:

- Message: UpvotePost
    - Verifies the existence of user, subreddit, and post.
    - Increments the post's upvote count and increases the author's karma.

- Message: DownvotePost
    - Similar validation as UpvotePost.
    - Increments downvote count and decreases the author's karma.

Messaging and Feeds:

- Message: SendMessage
    - Allows one user to send a direct message to another user.
    - The message, represented as a DirectMessage object (containing sender_id, content, and timestamp), is stored in the receiver's inbox.

- Message: GetDirectMessages
    - Retrieves and prints all messages in a user's inbox.

- Message: GetFeed
    - Fetches and displays posts from all subreddits to which the user is subscribed.
    - Provides a view of current activity in communities of interest.

Simulation

- **Function:** simulate_activity
    - Executes until the simulation time is exceeded.
    - At each iteration, selects a random user action from a predefined set: JoinSubReddit, LeaveSubReddit, CreatePost, CreateComment, CreateCommentReply, UpvotePost, DownvotePost, SendMessage, GetFeed and GetDirectMessages.
    - Ensures the selected action is valid and updates the state accordingly.
    - Once the simulation concludes, triggers PrintStats.

- Message: PrintStats
    - Displays overall simulation metrics including: Total Accounts, Total SubReddits, Total Posts, Total Comments, Total Messages, Total Upvotes and Total Downvotes.

Zipf Distribution Implementation:

Subreddit weights are calculated inversely proportional to their rank. These weights determine the likelihood of each subreddit being joined. The cumulative sum of all weights forms a selection range, and a random value within this range maps to a specific subreddit via the recursive helper function **find_subreddit_by_weight()**. This way, subreddits with

higher weights (popular ones) are chosen more often, while smaller subreddits are picked less frequently.

Subreddits with more members also generate more posts and some of these posts are randomly selected to be re-posts.

Performance and Observations:

The Reddit Clone simulator was evaluated under multiple configurations to measure its scalability and interaction dynamics as the number of users and subreddits increased. Each simulation recorded the total number of posts, comments, upvotes, downvotes, and direct messages generated during the run.

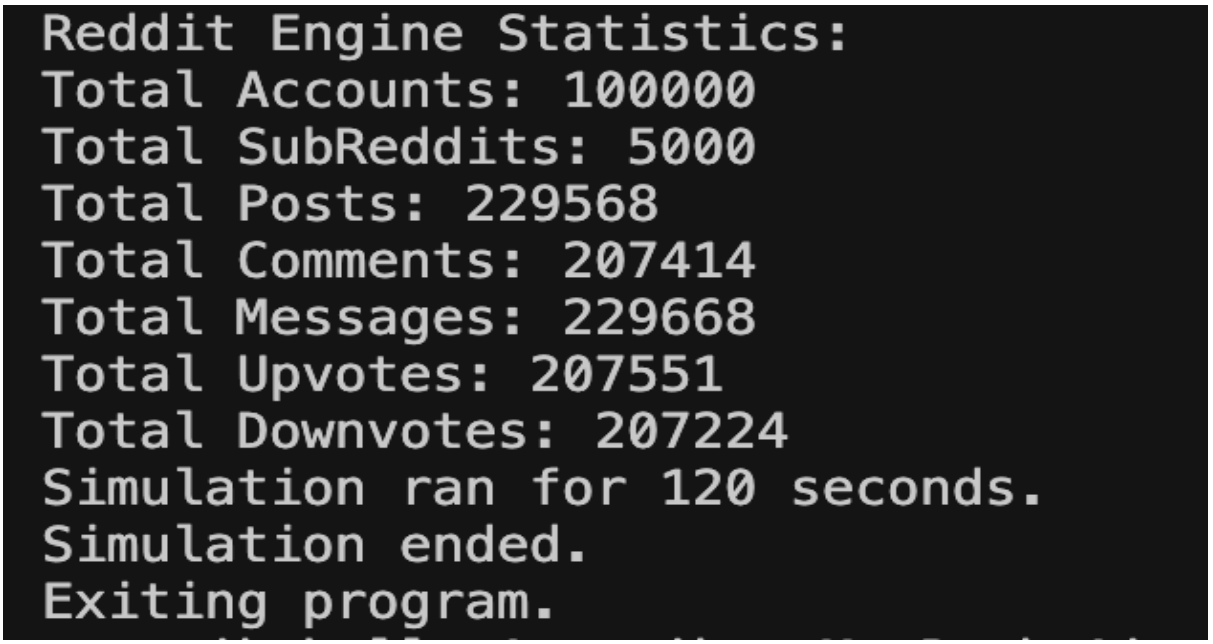
Observed Results:

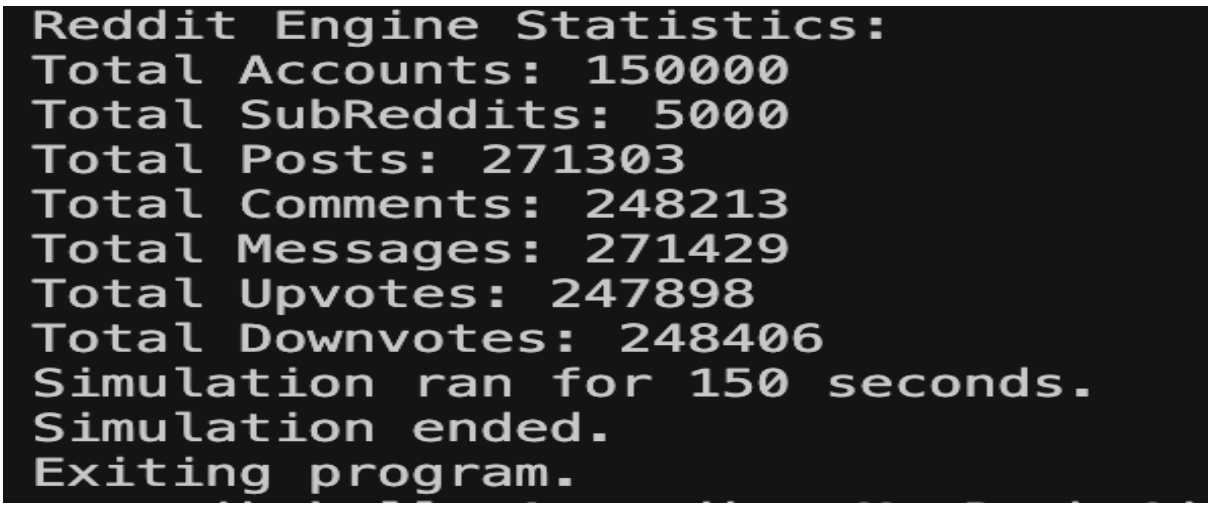
Users	Subreddits	Simulation Time (sec)	Total Posts	Total Comments	Total Upvotes	Total Downvotes	Total Messages
100,000	5,000	120	229,568	207,414	207,551	207,224	229,668
150,000	5,000	150	271,303	248,213	247,898	248,406	271,429

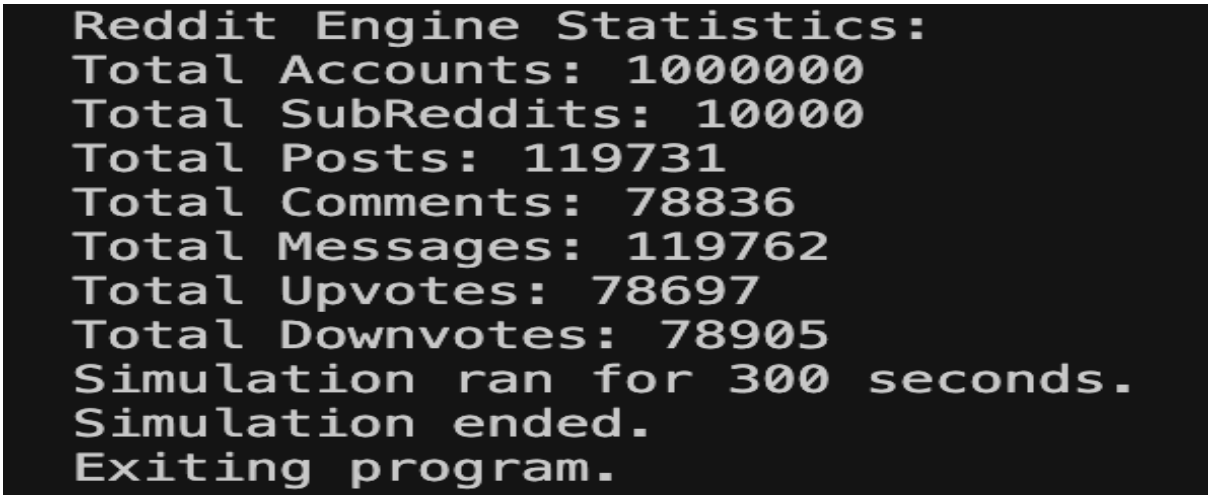
Engine Performance Metrics for 100,000 Users, 5,000 Subreddits, 120 sec simulation time:

Metric	Total	Per Second
Posts	229,568	1,913 posts/sec
Comments	207,414	1,728 comments/sec

Upvotes	207,551	1,730 upvotes/sec
Downvotes	207,224	1,727 downvotes/sec
Direct Messages	229,668	1,914 messages/sec







Even at 1,000,000 users and 10,000 subreddits, the reddit engine maintained functional correctness and stability throughout the 300-second simulation.

Part II:

In this part of the project, we implement a REST API interface for the Reddit Engine.

Features of the REST API Interface:

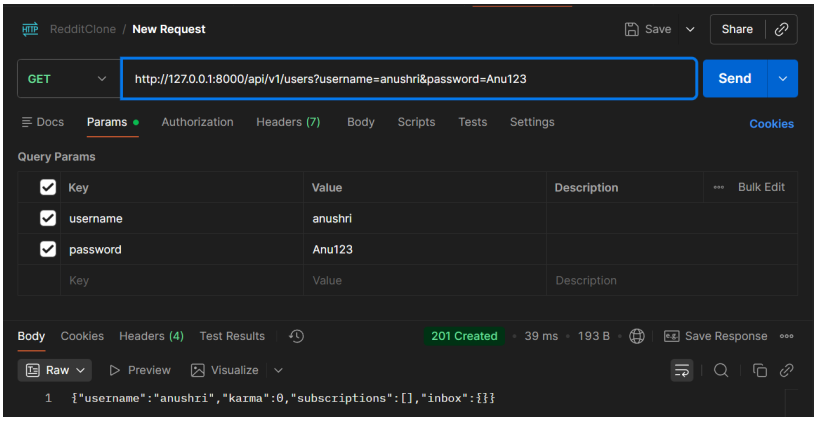
- API design similar to Reddit REST specification.
- Client actors making API requests to simulate user activity like creating a user, deleting a user, creating a subreddit, joining a subreddit, creating a post, upvoting, downvoting, commenting, and so on.
- Modified Simulator behavior to make REST calls to the engine.

REST APIs:

Create User:

API: /api/v1/users/

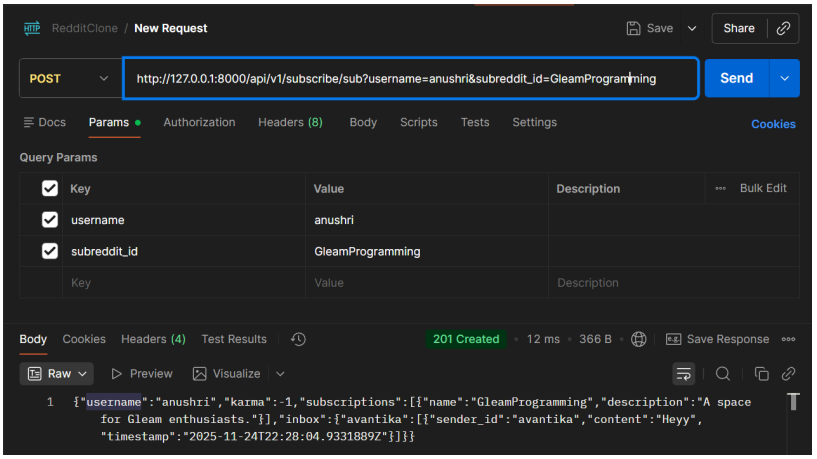
Function: Creates a new user account and returns the created account information.



Join Subreddit:

API: /api/v1/subscribe/sub/

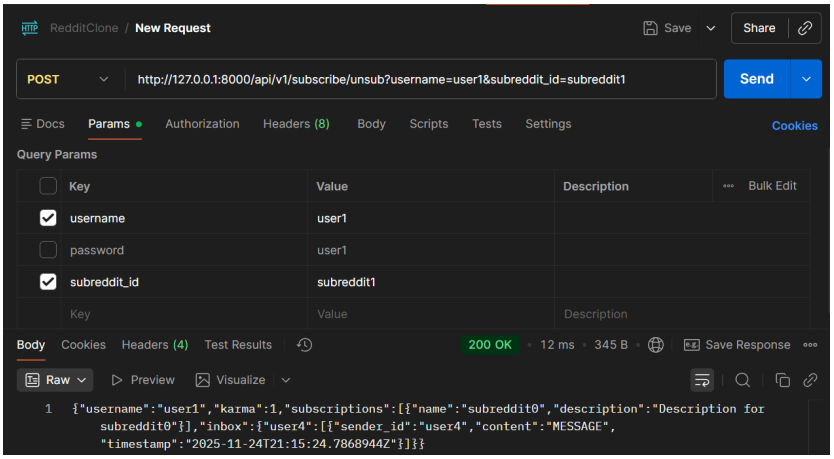
Function: User subscribes to the given subreddit.



Leave subreddit:

API: /api/v1/subscribe/unsub/

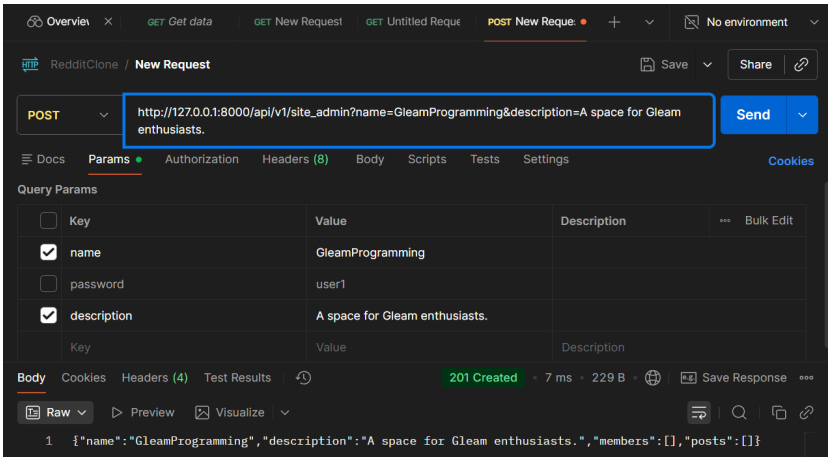
Function: User leaves or unsubscribes from the given subreddit.



Create Subreddit:

API: /api/v1/site_admin/

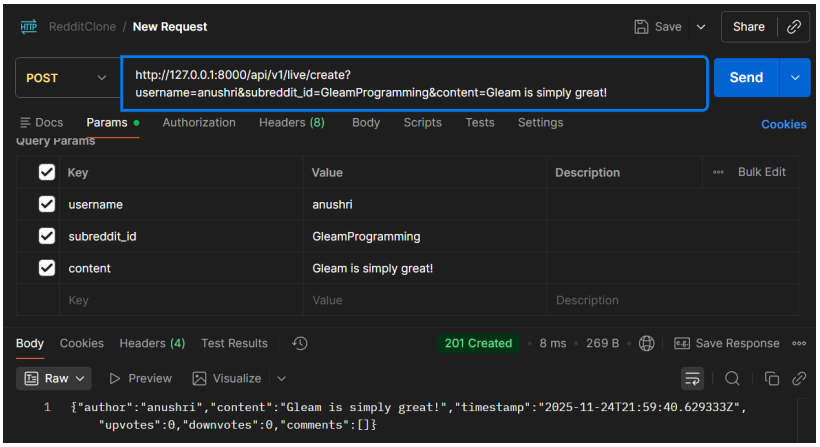
Function: To create a new subreddit.



Create Post:

API: /api/v1/live/create/

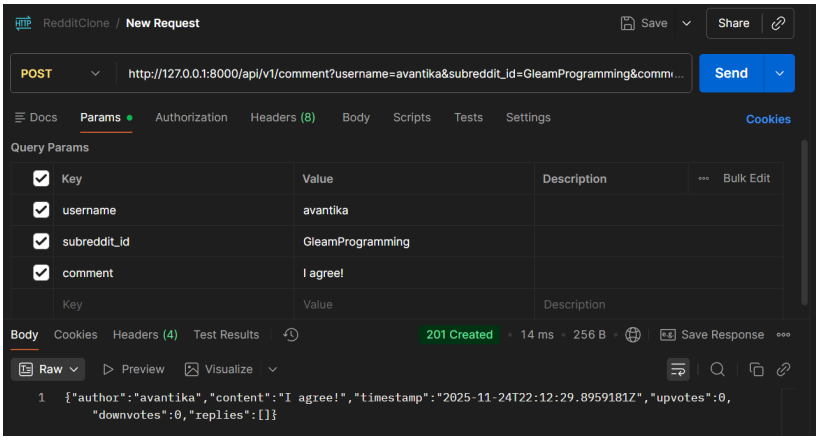
Function: Allows user to create a new post in a subreddit.



Create Comment:

API: /api/v1/comment/

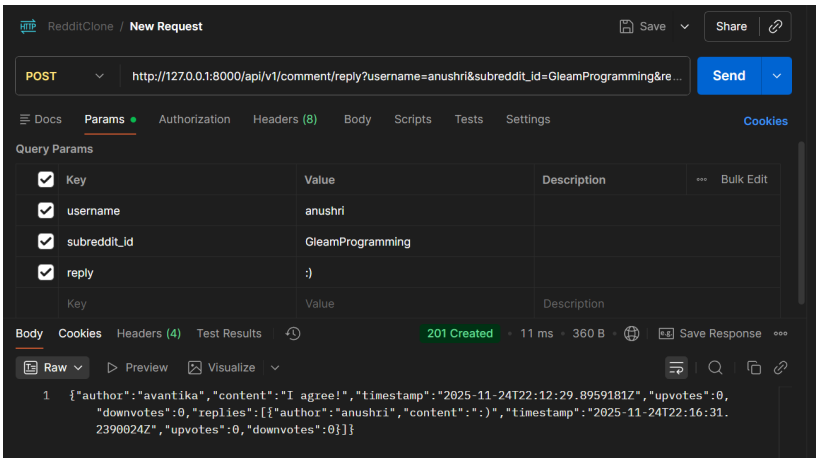
Function: Allows a user to create a comment on a post.



Create Comment Reply:

API: api/v1/comment/reply/

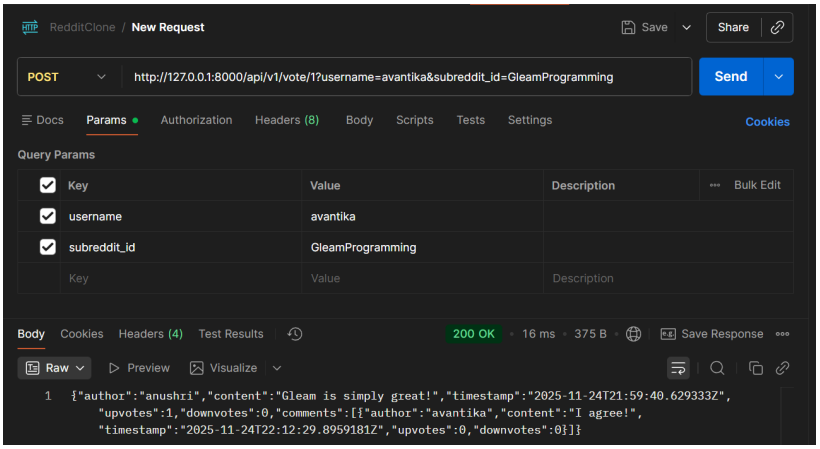
Function: Allows users to interact with other user comments under a post.



Upvote Post:

API: /api/v1/vote/1/

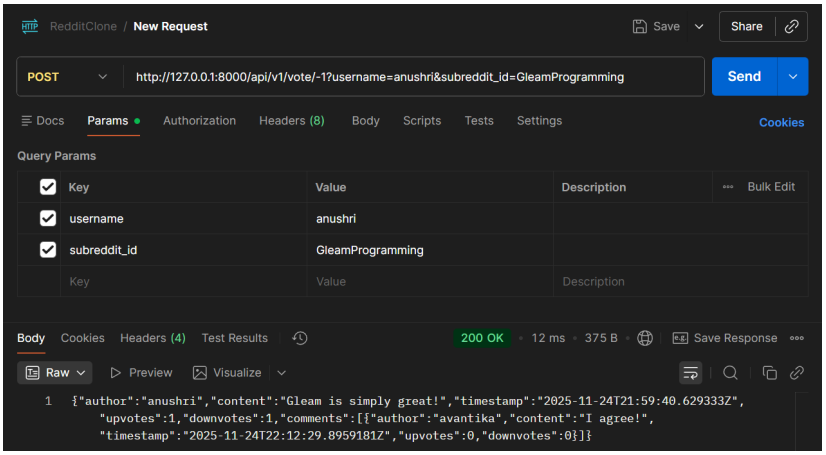
Function: Allows a user to upvote a post in a subreddit.



Downvote Post:

API: /api/v1/vote/-1/

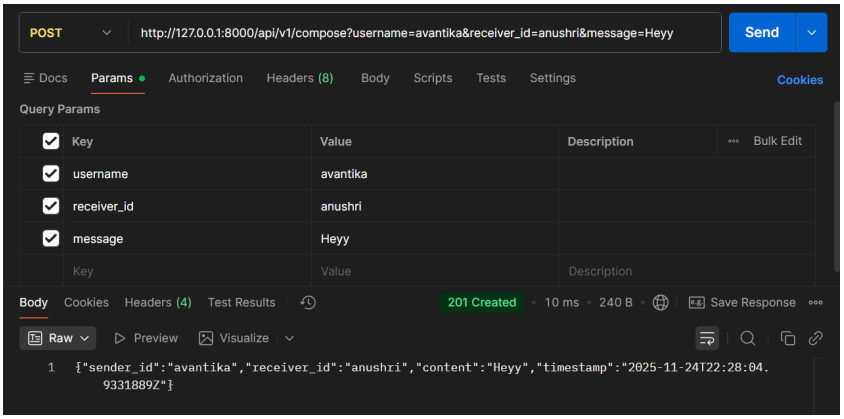
Function: Allows a user to downvote a post in a subreddit.



Send Direct Message:

API: /api/v1/compose/

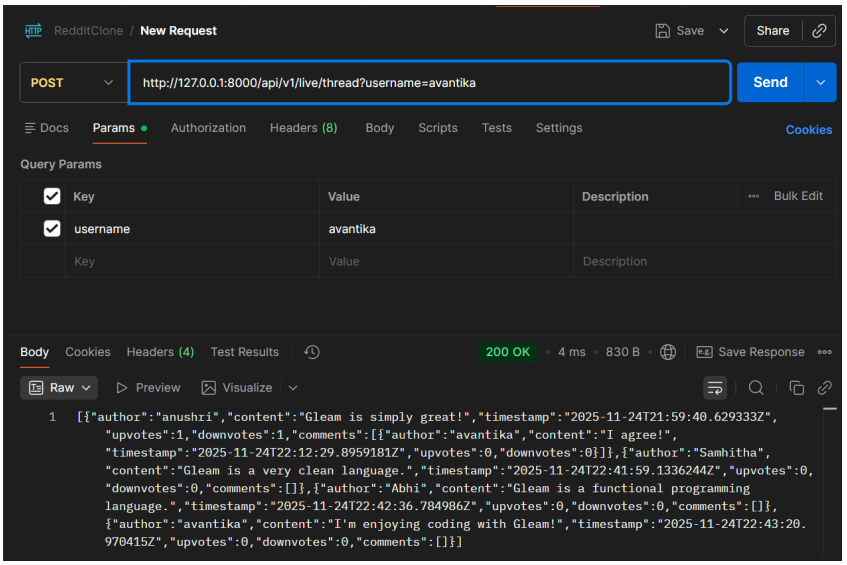
Function: Allows a user to send a direct message to another user.



Get Feed:

API: /api/v1/live/thread/

Function: User loads their feed to see posts from all their subscribed subreddits.



Get Inbox:

API: /api/v1/message/inbox/

Function: User gets an inbox full of all the messages they have received, segregated by the sender.